

02 · Instalación y ejecución

Documentacion del sistema

Version analizada: **0.1.0**

Commit analizado: **2282a33**

Fecha de generacion: **2026-08-27**

Generado a partir de 02-installation-and-execution.md — los Markdown son la fuente unica.
No editar este PDF: editar el Markdown y regenerar.

Contenido de este documento

1. Requisitos previos
2. Verificación del entorno
3. Obtener el código
4. Instalación de dependencias
5. Compilación
6. Ejecución
7. Configuración inicial
8. Base de datos
9. Permisos de macOS
10. Ejecución de pruebas
11. Réplica local de la integración continua
12. Empaquetado
13. Generación de la documentación en PDF
14. Errores frecuentes durante la instalación
15. Comandos verificados en este análisis

Cómo pasar de un repositorio recién clonado a un binario funcionando, y qué hacer cuando algo no compila o no devuelve datos. Todos los comandos de este documento se ejecutan desde la raíz del repositorio salvo que se indique otra cosa.

1. Requisitos previos

1.1 Sistema operativo

Requisito	Valor	Dónde está declarado
Sistema	macOS 13 (Ventura) o superior	<code>packaging/macos/Info.plist</code> → <code>LSMinimumSystemVersion</code>
Arquitectura	Apple Silicon (<code>aarch64</code>) o Intel (<code>x86_64</code>)	<code>.github/workflows/release-macos.yml</code>
Espacio en disco	~2 GB para <code>target/</code> en compilación de release	INFERENCIA , medido en una compilación limpia

El código compila y los tests pasan en versiones anteriores a Ventura, pero el `.app` declara 13.0 como mínimo y no hay verificación en CI sobre versiones antiguas: **REQUIERE VALIDACIÓN**.

1.2 Cadena de compilación

Herramienta	Versión mínima	Cómo obtenerla
Rust (<code>cargo</code> , <code>rustc</code>)	1.82 (<code>Cargo.toml</code> → <code>rust-version</code>)	https://rustup.rs
Componentes <code>rustfmt</code> y <code>clippy</code>	los del canal estable	<code>rust-toolchain.toml</code> los declara
Herramientas de línea de comandos de Xcode	las que instale <code>xcode-select</code>	<code>xcode-select --install</code>

`rust-toolchain.toml` fija `channel = "stable"`, así que `rustup` selecciona la toolchain correcta automáticamente al entrar en el directorio.

Rust por Homebrew. Funciona para compilar y probar, pero **no** para el binario universal: `brew install rust` trae solo el target del host. `scripts/package-app.sh` lo detecta y se degrada a la arquitectura nativa con un aviso, en vez de fallar.

1.3 Utilidades de macOS

Todas vienen con el sistema. `scripts/verify-environment.sh` comprueba estas once: `/usr/sbin/lsof`, `/usr/sbin/spctl`, `/usr/bin/csrutil`, `/usr/bin/fdesetup`, `/usr/bin/codesign`, `/usr/sbin/arp`, `/sbin/route`, `/sbin/ifconfig`, `/bin/launchctl`, `/bin/ps` y `/usr/bin/log`. Si falta alguna, la superficie correspondiente queda vacía y el producto lo declara; no aborta.

1.4 Herramientas opcionales

Herramienta	Para qué	Sin ella
<code>iconutil</code>	Generar <code>AppIcon.icns</code> en el <code>.app</code>	<code>package-app.sh</code> falla
<code>hdiutil</code>	Crear el <code>.dmg</code>	<code>package-dmg.sh</code> falla

<code>python3</code>	Generar el icono y los PDF de documentación	Icono y PDF no se generan
<code>gh</code> (GitHub CLI)	Publicar la release con <code>release-product.sh --publish</code>	Solo se construye <code>dist/</code>

2. Verificación del entorno

```
./scripts/verify-environment.sh
```

Informa, no instala. Devuelve `1` si falta algún requisito imprescindible y `0` si el entorno está listo (aunque haya avisos). Comprueba sistema, cadena de compilación, utilidades, herramientas de empaquetado, si el terminal tiene Acceso total al disco y si se está ejecutando como root.

3. Obtener el código

```
git clone https://github.com/vladimiracunadev-create/rootcause-macos-inspector.git
cd rootcause-macos-inspector
```

4. Instalación de dependencias

No hay paso de instalación separado: `cargo` descarga y compila las dependencias declaradas en `Cargo.toml`, con las versiones exactas fijadas en `Cargo.lock` (326 paquetes transitivos). `rusqlite` usa la característica `bundled`, así que **compila SQLite desde el código fuente** y no depende de la librería del sistema; eso alarga la primera compilación.

5. Compilación

5.1 Edición completa (GUI + CLI)

```
cargo build --release
```

Produce `target/release/rootcause`. Es la edición por defecto: la feature `gui` está activa e incluye `eframe` y `egui`.

5.2 Edición CLI-only

```
cargo build --release --no-default-features
```

El mismo binario sin interfaz gráfica ni dependencias de ventana. Útil para servidores, sesiones SSH y contenedores de compilación. `src/main.rs` cambia de comportamiento: sin argumentos imprime la ayuda en vez de abrir una ventana.

5.3 Compilación de desarrollo

```
cargo build
cargo run -- status
```

Mucho más rápida de compilar y bastante más lenta de ejecutar. El perfil de release usa `opt-level = 3`, `lto = true`, `codegen-units = 1` y `strip = true`.

6. Ejecución

6.1 Interfaz gráfica

```
./target/release/rootcause
./target/release/rootcause --gui # equivalente explícito
```

La ventana abre con 1280 × 800 px y un mínimo de 880 × 600 px para que quepa en un MacBook de 13 pulgadas. La primera captura se pide antes de dibujar el primer frame.

6.2 Línea de comandos

```
./target/release/rootcause status      # veredicto completo
./target/release/rootcause status --json  # lo mismo, para encadenar
./target/release/rootcause --help      # los 19 comandos
```

Referencia completa de comandos en [05 · Referencia técnica](#) y en [docs/COMMANDS.md](#).

6.3 Instalación en el PATH

```
cargo install --path .          # instala en ~/.cargo/bin/rootcause
```

NO DOCUMENTADO EN EL REPOSITORIO: el `README.md` no menciona `cargo install`; funciona porque `Cargo.toml` declara el binario `rootcause`, pero `publish = false` impide publicarlo en crates.io.

7. Configuración inicial

No hace falta ninguna: el producto arranca con valores por defecto razonables y **no crea el archivo de configuración hasta que se lo pides**.

```
./target/release/rootcause config init  # crea el JSON con los valores por defecto
./target/release/rootcause config show  # muestra rutas y configuración efectiva
```

Ubicación: `~/Library/Application Support/RootCauseInspector/rootcause-config.json`. Detalle campo por campo en [10 · Configuración](#).

8. Base de datos

Tampoco requiere pasos: `PersistenceStore::new` crea el directorio de datos y ejecuta `ensure_schema()` en cada arranque, con sentencias `CREATE TABLE IF NOT EXISTS`. No hay sistema de migraciones ni scripts SQL externos.

Elemento	Ruta
Historial SQLite	<code>~/Library/ Application Support / RootCauseInspector /rootcause- history.db</code>
Estado del agente	<code>~/Library/ Application Support / RootCauseInspector /rootcause-agent -state.json</code>
Configuración	<code>~/Library/ Application Support / RootCauseInspector /rootcause- config.json</code>

Copia del historial	~/Library/ Application Support / RootCauseInspector /rootcause- history-backup. json
---------------------	--

Para empezar de cero basta con borrar esos archivos: se recrean solos. Esquema completo en [07 · Base de datos](#).

9. Permisos de macOS

RootCause funciona sin conceder nada, pero dos superficies quedan limitadas:

Permiso	Qué habilita	Cómo se concede	Si falta
Acceso total al disco	Leer <code>TCC.db</code> (sección Privacidad)	Ajustes del Sistema → Privacidad y seguridad → Acceso total al disco → añadir el binario o el <code>.app</code>	<code>TccOverview::readable = false</code> y una alerta explicativa
Automatización	Consultar login items vía System Events	Diálogo del sistema al pulsar «Consultar login items»	La lista de login items queda vacía

El permiso de Acceso total al disco hay que concederle al **binario concreto** que se ejecuta: darlo a `Terminal.app` cubre las ejecuciones desde ese terminal, pero no al `.app` empaquetado, que es otro ejecutable.

Ejecutar como root (`sudo rootcause status`) amplía lo que ve `lsOf` a los sockets de todos los usuarios. No es necesario y el producto no lo pide: `macos::environment` informa del contexto en vez de escalar.

10. Ejecución de pruebas

```
cargo test --all-features           # los 112 tests
cargo test --all-features -- --nocapture # igual, mostrando stdout (lo que hace la CI)
cargo test services::rules          # solo un módulo
```

Los tests son unitarios y viven junto al código en módulos `#[cfg(test)]`. Solo uno toca el sistema real:

`services::macos::tests::el_entorno_real_encuentra_las_utilidades_base_de_macos`, que comprueba que `launchctl` existe. Detalle en [12 · Pruebas y calidad](#).

11. Réplica local de la integración continua

```
./scripts/ci-local.sh
```

Ejecuta, en el mismo orden que `.github/workflows/ci.yml`: versiones, `cargo fmt --check`, `cargo clippy --all-targets --all-features -- -D warnings`, `cargo test --all-features`, build CLI-only, build completa y humo de la CLI. Si pasa en local, pasa en la CI.

La validación de Markdown de la CI es un job aparte que no cubre este script:

```
npm --yes markdownlint-cli2
```

12. Empaquetado

```
./scripts/package-app.sh           # dist/RootCause.app para la arquitectura nativa
./scripts/package-app.sh --universal # binario universal arm64 + x86_64 (necesita rustup)
./scripts/package-dmg.sh           # dist/RootCause-0.1.0.dmg y dist/SHA256SUMS.txt
```

`package-dmg.sh` construye el `.app` antes si no existe, y crea la imagen en un directorio temporal del disco interno porque `hdiutil` solo trabaja sobre APFS o HFS+: si el repositorio vive en un volumen exFAT o de red, crear la imagen ahí falla con «Operación no permitida».

Release completa en un comando:

```
./scripts/release-product.sh # construye dist/ y para ahí
./scripts/release-product.sh --publish --watch # además etiqueta, publica y espera al workflow
```

Detalle en [13 · Despliegue y operación](#).

13. Generación de la documentación en PDF

```
python3 -m pip install markdown xhtml2pdf
python3 scripts/build-docs-pdf.py # todos los documentos
python3 scripts/build-docs-pdf.py 07 11 # solo los que empiecen por 07 y 11
python3 scripts/build-docs-pdf.py --check # solo comprueba dependencias
```

Si prefieres no tocar el Python del sistema, un entorno virtual funciona igual:

```
python3 -m venv .venv && .venv/bin/pip install markdown xhtml2pdf
.venv/bin/python scripts/build-docs-pdf.py
```

14. Errores frecuentes durante la instalación

Síntoma	Causa	Solución
<code>error: linker 'cc' not found</code>	Faltan las herramientas de Xcode	<code>xcode-select --install</code>
<code>error: failed to run custom build command for libsqlite3-sys</code>	Compilador de C no disponible o SDK incompleto	Reinstalar herramientas de Xcode; <code>rustup update</code>
<code>error: package requires rustc 1.82</code>	Toolchain antigua	<code>rustup update stable</code>
<code>lipo: can't open input file ... x86 _64-apple-darwin /release/ rootcause</code>	Rust instalado por Homebrew, sin el segundo target	Instalar Rust con <code>rustup</code> , o construir sin <code>--universal</code>
<code>hdiutil: create failed - Operation not permitted</code>	El repositorio está en exFAT o en red	Ya lo evita <code>package-dmg.sh</code> ; si aparece, ejecutar desde un directorio del disco interno
Ventana en negro o error de <code>glow</code> al abrir la GUI	Contexto gráfico no disponible (sesión SSH, sin pantalla)	Usar la edición CLI-only o un comando del CLI
<code>iconutil: command not found</code>	Herramientas de Xcode incompletas	<code>xcode-select --install</code>
La sección Privacidad aparece vacía y avisa	Falta Acceso total al disco	Concederlo al binario o al <code>.app</code> (sección 9)
<code>No se observaron conexiones</code>	<code>lssof</code> sin privilegios solo ve los sockets propios	Es el comportamiento esperado; ejecutar con <code>sudo</code> si se necesita la vista completa
Archivos <code>._*</code> en <code>dist/</code>	Volumen exFAT que crea bifurcaciones AppleDouble	<code>export COPYFILE_DISABLE=1</code> , que ya hace <code>release-product.sh</code>

15. Comandos verificados en este análisis

Ejecutados en el commit analizado, sobre macOS 26.3.1 (build 25D2128), Apple Silicon (arm64):

Comando	Resultado
<code>cargo fmt --all</code> <code>-- --check</code>	Sin diferencias (código de salida 0)
<code>cargo clippy --</code> <code>all-targets --</code> <code>all-features --</code> <code>-D warnings</code>	Sin advertencias (código de salida 0)
<code>cargo test --all</code> <code>-features</code>	112 passed; 0 failed; 0 ignored, 1,47 s
<code>git diff --stat</code> <code>-- src/</code>	71 inserciones, 0 eliminaciones (solo comentarios ///)

Los mismos cuatro pasos los repite la CI en `macos-latest` en cada push, más las dos compilaciones de release y el humo de la CLI.

Siguiente lectura recomendada: [03 · Arquitectura](#).